



LOOPING

Chapter 5



Introduction

- We use loop when you want to execute statement several times until a condition is reached.
- Generally, loops consist of two parts:
 - ☐ One or more control expressions which control the execution of the loop.
 - ☐ Body , which is the statement or a set of statement which is executed over and over



3 types of loop

- For loop
- While loop
- Do while loop



For loops

- The basic format of the for statement is,

```
for( start_condition; continue_ condition; re-evaluation ) {  
    program statement;  
}
```



Sample program

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
int count;
```

```
for(count = 1; count <= 10; count=count+1)
```

```
printf("%d ", count);
```

```
}
```

Sample Program Output 1 2 3 4 5 6 7 8 9 10



Example

// An example of using a for loop to print out characters

```
#include <stdio.h>
main()
{
    char letter;
    for( letter = 'A'; letter <= 'E'; letter = letter + 1 )
        printf("%c", letter);
}
```

- **Sample Program Output A B C D E**



Sum of the first n numbers

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
int i,n,sum=0;
```

```
printf("ENTER THE LIMIT\n");
```

```
scanf("%d",&n);
```

```
for(i=1;i<=n;i++)
```

```
{
```

```
sum=sum+i;
```

```
}
```

```
printf("\n THE SUM OF FIRST %d NUMBERS = %d", n, sum);
```

```
}
```



MULTIPLICATION TABLE

```
#include<stdio.h>
main()
{
    int i, table, counter=1;

    printf("ENTER THE MULTIPLICATION TABLE \n");
    scanf("%d",&table);
    printf("ENTER THE COUNTER VALUE\n");
    scanf("%d",&counter);
    printf("THE REQUIRED TABLE IS SHOWN BELOW:\n");
    for(i=0;i<=counter;i++)
        printf("\n%d * %d = %d",table,i,i*table);
}
```



Multiplication table from 1 to 10

```
#include<stdio.h>
main()
{
int i,j;
for(i=1; i<=10; i++)
{
for(j=1; j<=10; j++)
{
printf("%d * %d = %d\n" ,i,j,i*j);
}
printf("\n\n");
printf("Multiplication table of %d\n" ,i);
printf("\n\n");
}
}
```



Inside the other loop

- Sometime any loop could be placed inside the other loop

```
#include<stdio.h>
main()
{
    int i,j;
    for(i=1;i<=5;i++)
    {
        for(j=1;j<=i;j++)
        printf("%d\t",j);
        printf("\n");
    }
}
```



The output is:

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5



Example

```
#include<stdio.h>

main(){
    int i,j;
    for(i=1;i<=5;i++)
    {
        for(j=5;j>=i; j--)
        printf("%d\t",j);
        printf("\n");
    }
}
```

- The output is:

```
5  4  3  2  1
5  4  3  2
5  4  3
5  4
5
```



While loop

- The while provides a mechanism for repeating C statements whilst a **condition is true**.
- While loop has one control expression and executes as long as that expression is true.
- General structure (its format is):

```
while(expression)
{
statements;
}
```



While Loop

```
/*Sample program including While loop*/  
#include<stdio.h>  
main()  
{  
  
    int loop = 0;  
    while(loop <= 10)  
    {  
        printf ("%d\n",loop);  
        ++loop;  
    }  
}
```



Multiplication table from 1 to 10

```
#include<stdio.h>
main(){
int i,j;i=1;
while(i<=10){
j=1;
while(j<=10){
printf("%d x %d = %d\n",i,j,i*j);
j++;
}
printf("\n\n");i++;
}
}
```



do while

- When developing programs, it sometimes become desirable to have the **test made at the end** of the loop rather than at the beginning.
- This looping statement is known as the **do statement**.

- The syntax is:

```
do{  
statements;  
}  
while(expression);{  
statements;  
}
```



Do While

- Execution of the do statement proceeds as follows
 - Program statement is executed first.
 - Next, the expression inside the parenthesis is evaluated, if the result of evaluating is **TRUE**, the loop continues and the program statement is once again executed.
 - As long as evaluation of expression continues to be true, program statement is repeatedly executed.
 - When evaluation of the expression is **FALSE**, the loop is terminated and the next statement in the program is executed in the normal sequential manner.



Do While

- Execution of the do statement proceeds as follows(cont'd)
 - The do statement is simply a transposition of the while loop statement with the looping conditions placed at the end of the loop rather than at the beginning.
 - **N.B:** Remember that, unlike the for and while loops, the do statement guarantees that the body of the loop will be executed at least once.



Do while

```
/* Program that displays number from 1 to 9 using Do
   while */
#include<stdio.h>
main(){
int i;
i=1;
do{
printf("\n i is:%d",i);
i=i+1;
}
while(i<10);
}
```



Exercise

- Make a program to print out multiplication table of 7 using do while.

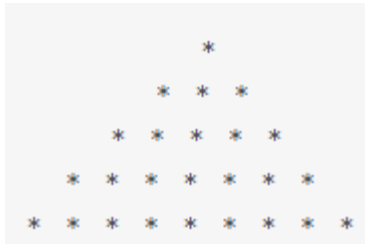


Answer

```
#include<stdio.h>
main(){
int i,j;
i=7,j=1;
do{
    printf ("\n %d  x  %d=  %d", i, j, i*j);
    j++;
}
while ( i<=10 );
}
```



Pyramid program using *

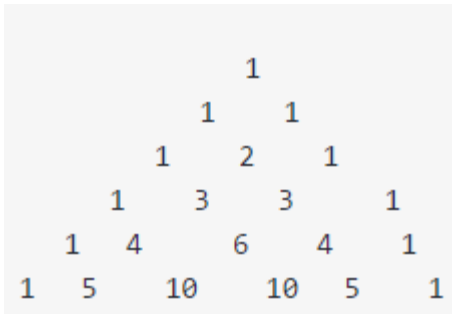


```
#include <stdio.h>
int main()
{
    int i, space, rows, k=0;
    printf("Enter number of rows: ");
    scanf("%d",&rows);
    for(i=1; i<=rows; ++i, k=0)
    {
        for(space=1; space<=rows-i; ++space)
        {
            printf(" ");
        }
        while(k != 2*i-1)
        {
            printf("*");
            ++k;
        }
        printf("\n");
    }

    return 0;
}
```

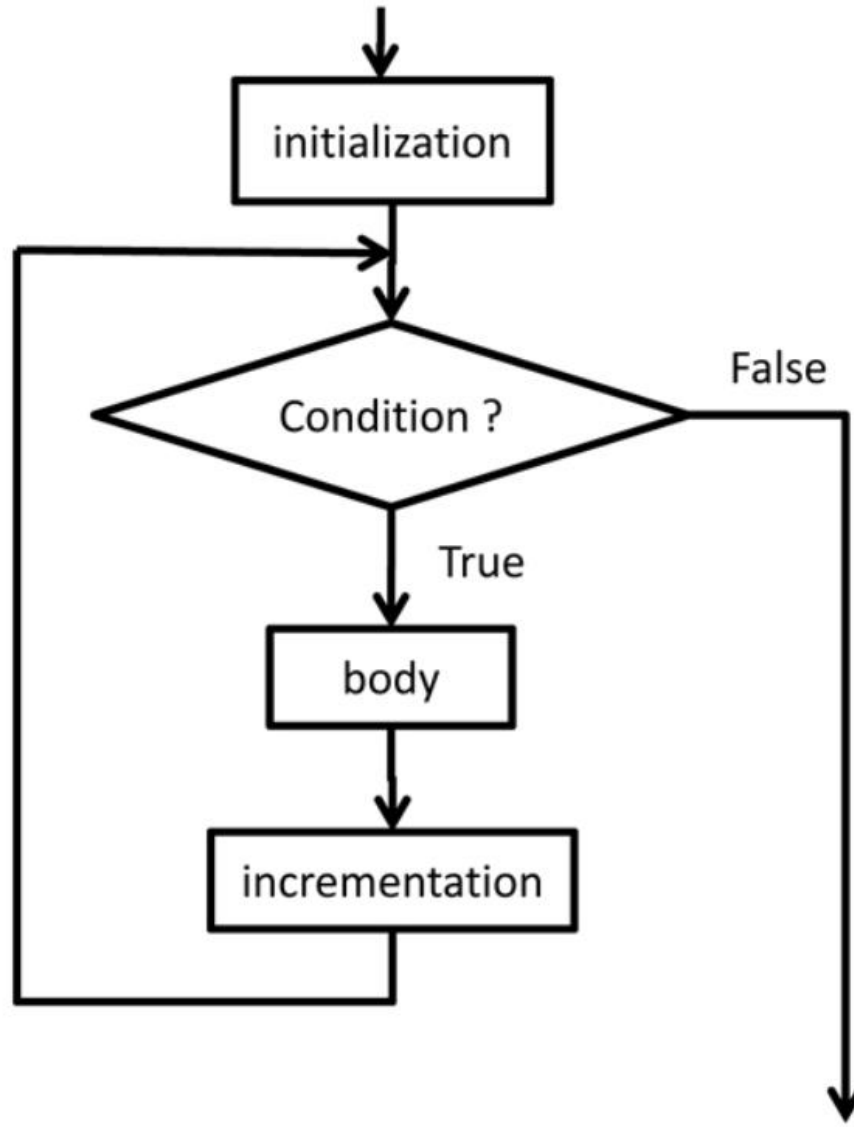


Pascal's Triangle



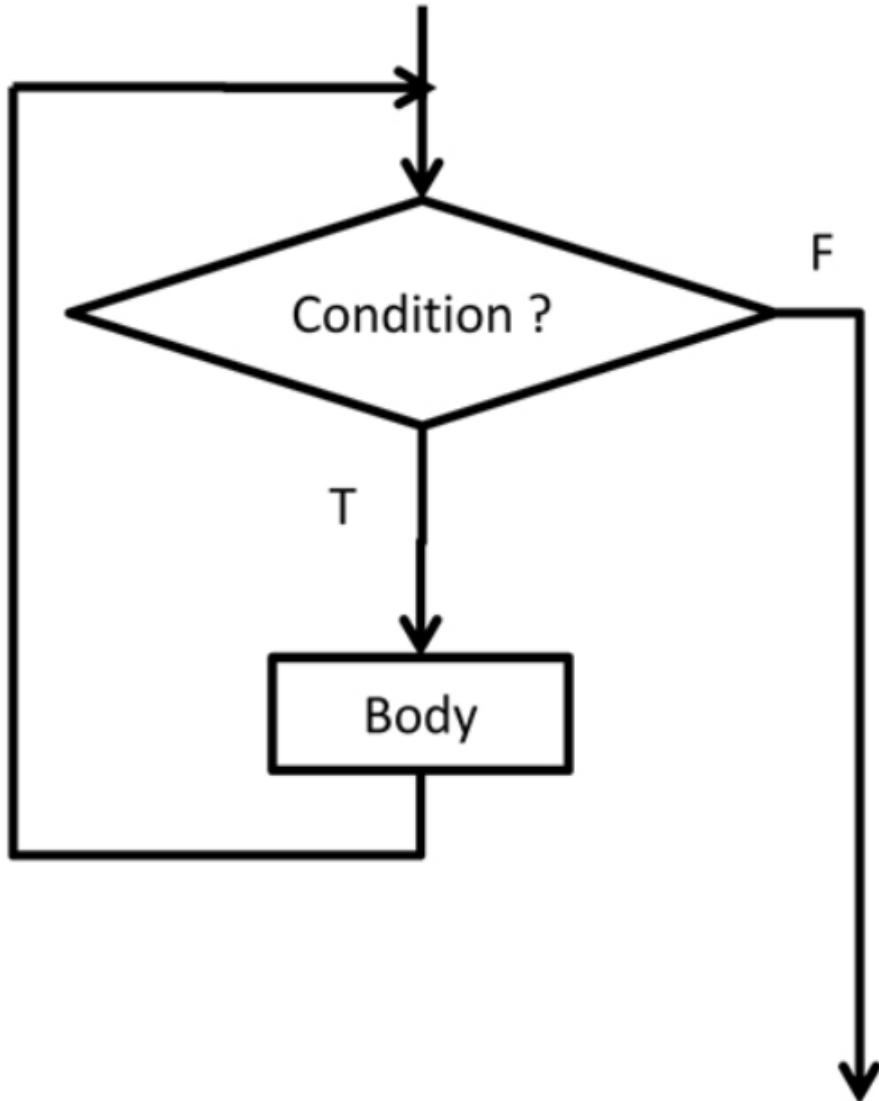
```
#include <stdio.h>
int main()
{
    int rows, coef = 1, space, i, j;
    printf("Enter number of rows: ");
    scanf("%d",&rows);
    for(i=0; i<rows; i++)
    {
        for(space=1; space <= rows-i; space++)
            printf(" ");
        for(j=0; j <= i; j++)
        {
            if (j==0 || i==0)
                coef = 1;
            else
                coef = coef*(i-j+1)/j;
            printf("%4d", coef);
        }
        printf("\n");
    }
    return 0;
}
```

For(initialization; condition;
incrementation) body;



While versus Do-While Loops

while(condition)
body;



do {
body;
} while(condition);

